

Object-Oriented Programming in Java

Emily Lucia Antosch

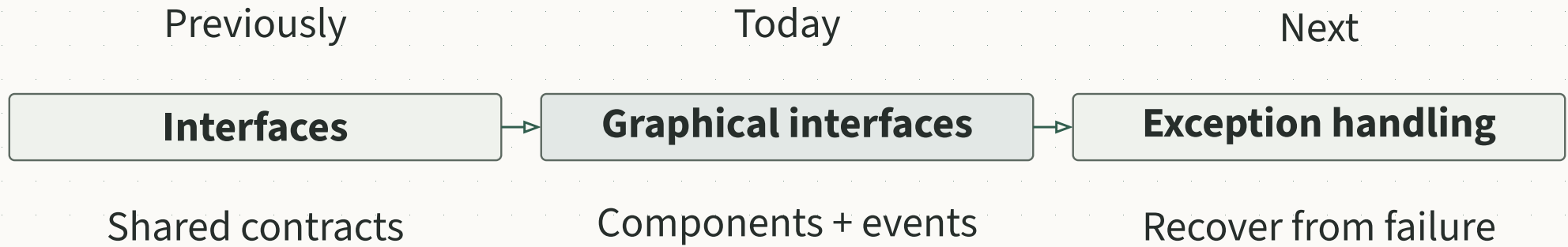
18.09.2026

Today's route

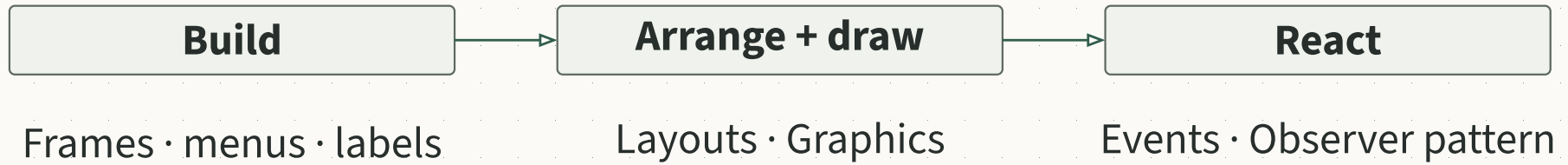
1. Introduction	2
2. Basic Structure	5
3. Creating Graphical User Interfaces	10
4. Layout	20
5. Drawing	28
6. Buttons & Events	34
7. Simple Dialogs	47
8. Suggestions	50
9. License Notice	52

1. Introduction

From Contracts to User Interfaces



What You Will Build

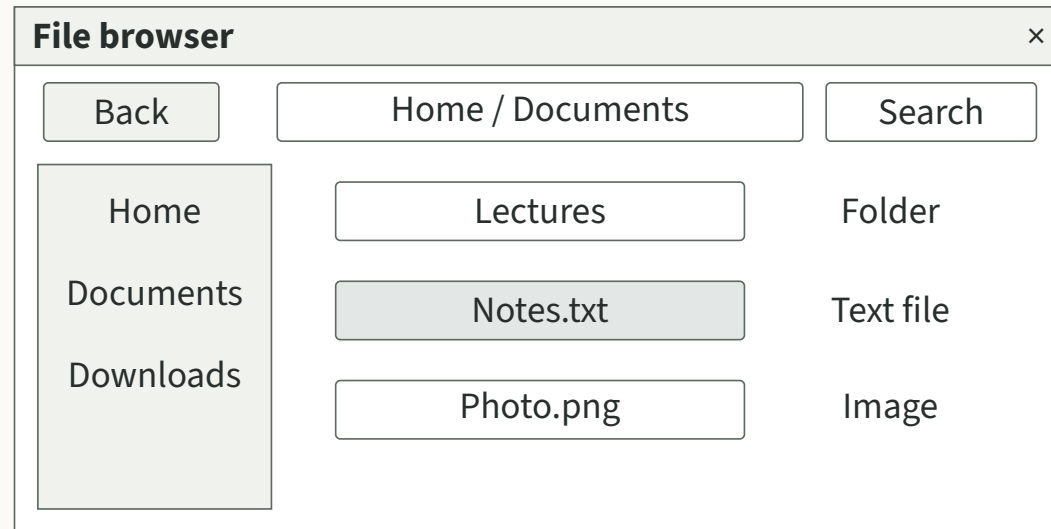


2. Basic Structure

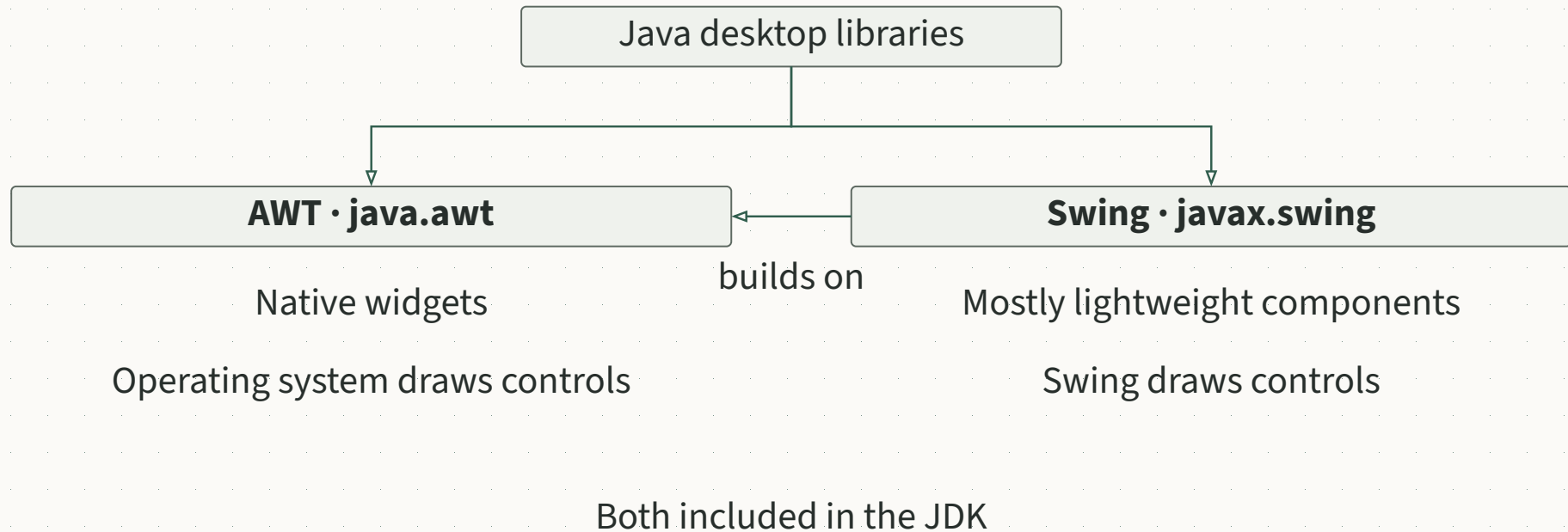
Read a Graphical Interface



Which controls do you recognize? How does one control affect another?

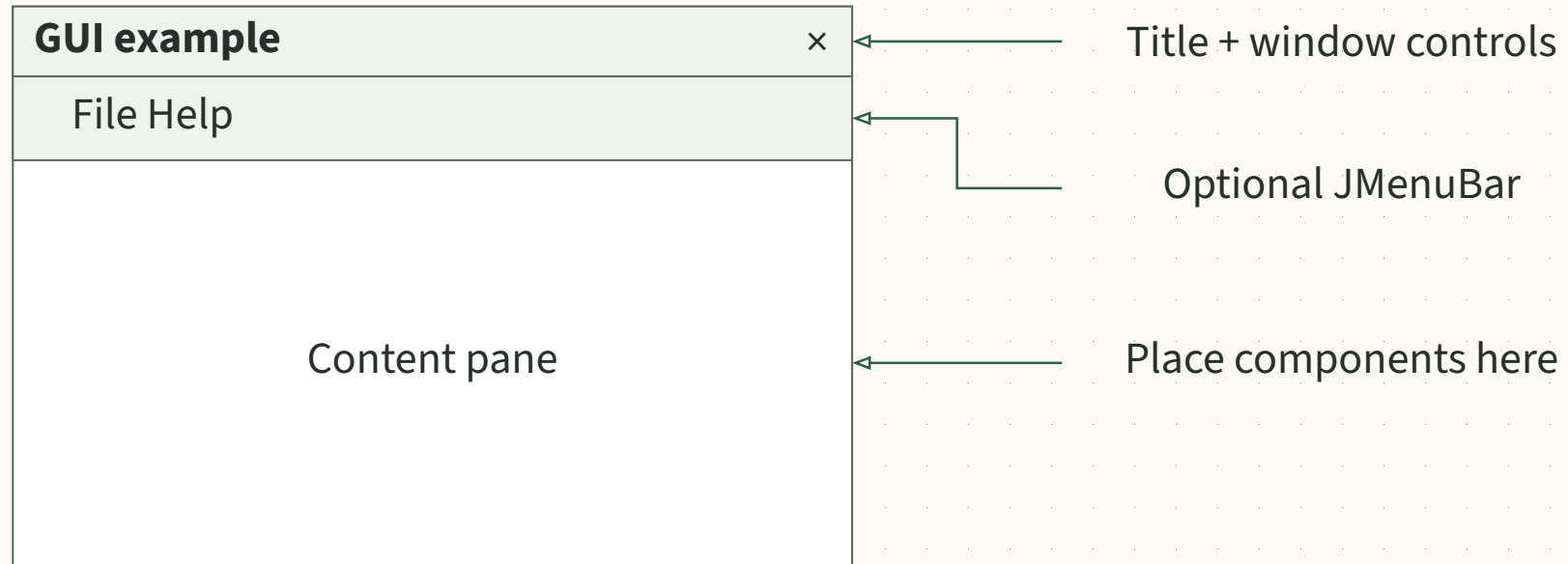


AWT and Swing



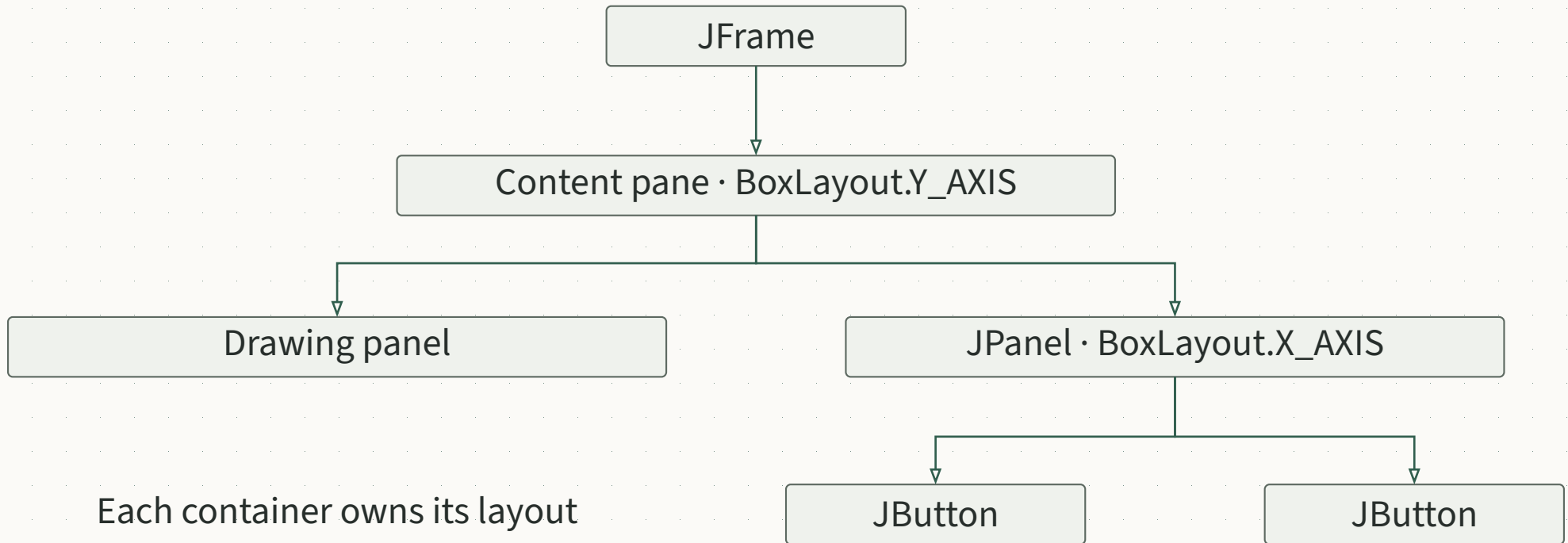
Swing uses AWT events, graphics and top-level windows.

Anatomy of a Frame



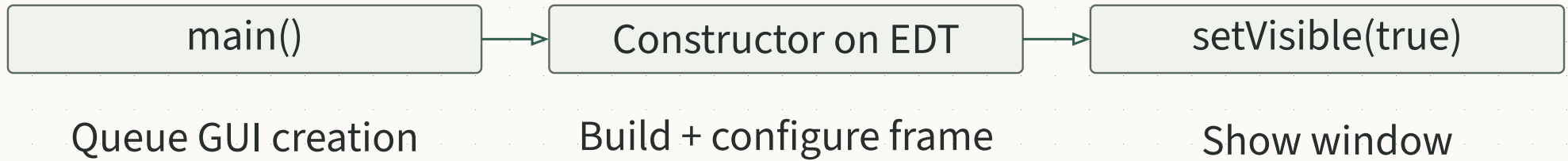
JFrame contains both menu bar and content pane

Components Form a Tree



3. Creating Graphical User Interfaces

Create, Configure, Show



Swing components are created and updated on the **event dispatch thread (EDT)**. The following examples omit imports (`javax.swing.*`, `java.awt.*`, `java.awt.event.*`).

A Minimal Window

```
1  public class HelloWorld {
2      public HelloWorld() {
3          JFrame frame = new JFrame("GUI example");
4          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
5          frame.setVisible(true);
6      }
7      public static void main(String[] args) {
8          SwingUtilities.invokeLater(HelloWorld::new);
9      }
10 }
```

Java

Use the same EDT startup pattern for the later GUI classes.

Size and Position



What happens without `EXIT_ON_CLOSE`? Why call `setVisible(true)`?



Task 1

Make the frame 400×300 pixels. Place its top-left corner at (50, 50).

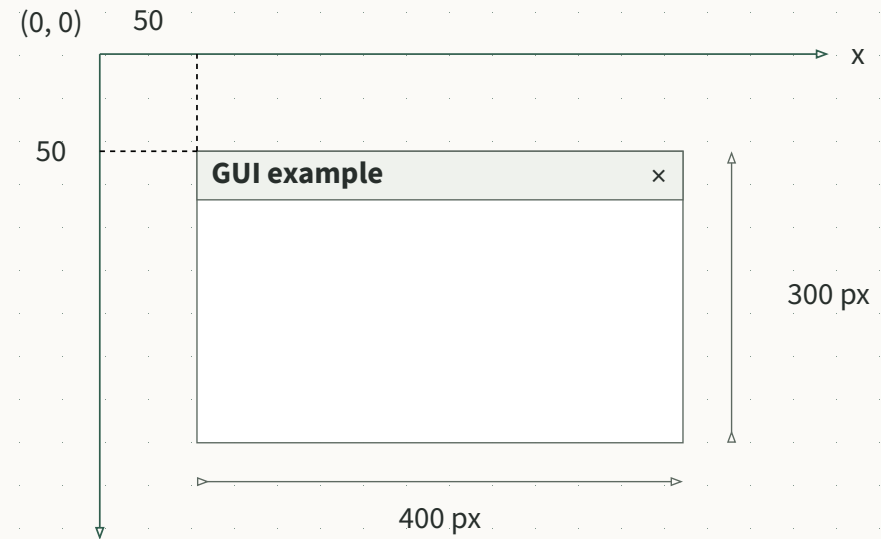
Inspect `JFrame`'s methods. Set size and location before showing the frame.

Size and Position

```
1 frame.setSize(400, 300);  
2 frame.setLocation(50, 50);  
3 frame.setVisible(true);
```

Java

A new JFrame is initially hidden. The default close operation hides it; it does not exit the application.



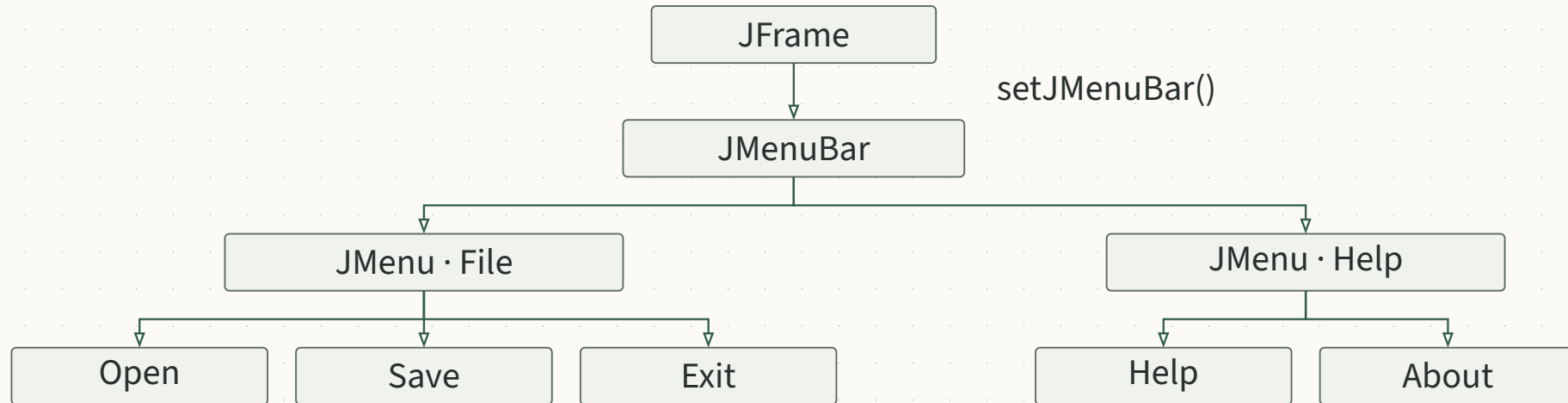
Frame size includes borders and title bar · not to scale

Menus Are Components Too



Task 2

Add File → Open, Save, Exit and Help → Help, About.



Bottom row: `JMenuItem` · menus and items use `add()`

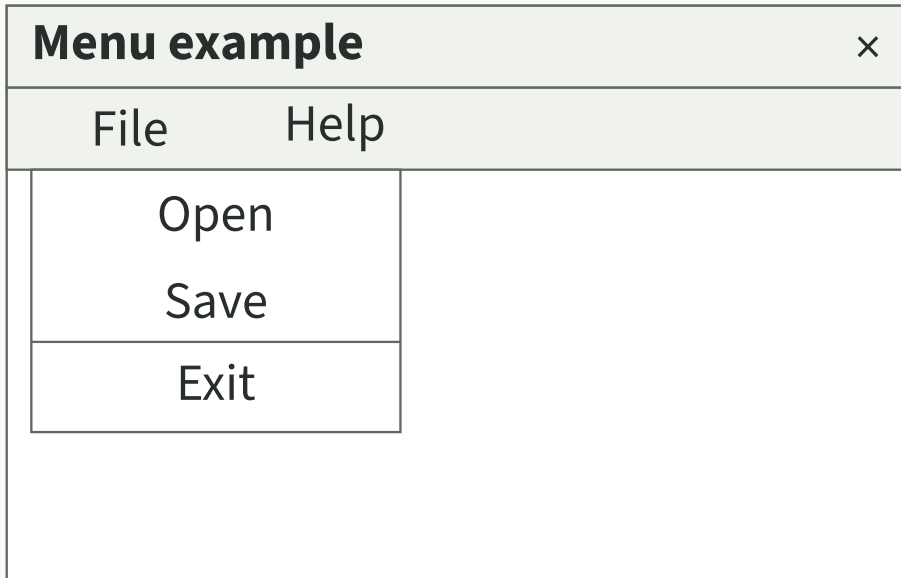
Build the Menu Bar

```
1 JMenuBar menuBar = new JMenuBar();
2 frame.setJMenuBar(menuBar);
3 JMenu menuFile = new JMenu("File");
4 menuBar.add(menuFile);
5 menuFile.add(new JMenuItem("Open"));
6 menuFile.add(new JMenuItem("Save"));
7 menuFile.addSeparator();
8 menuFile.add(new JMenuItem("Exit"));
```

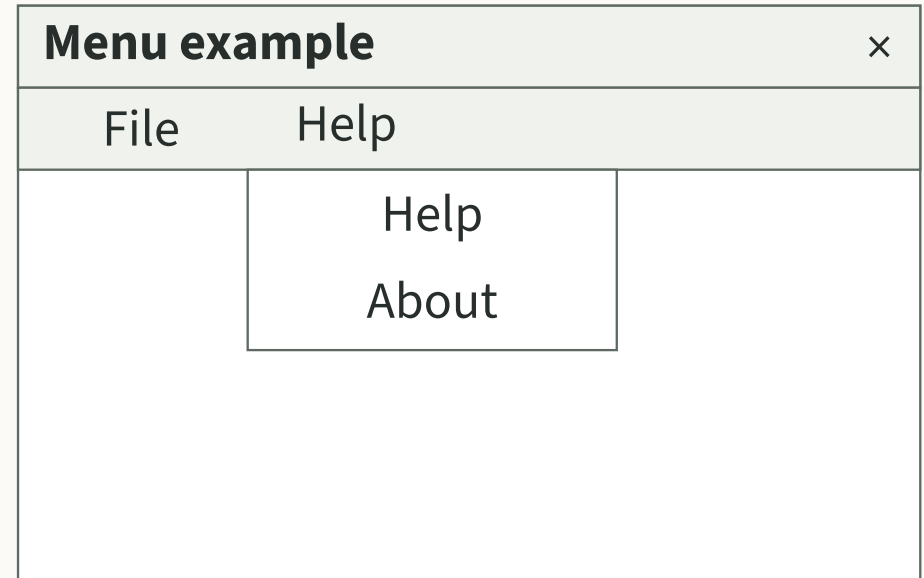
Java

Add the Help menu in the same way, then show the frame. Menu items need listeners before they perform an action.

Build the Menu Bar

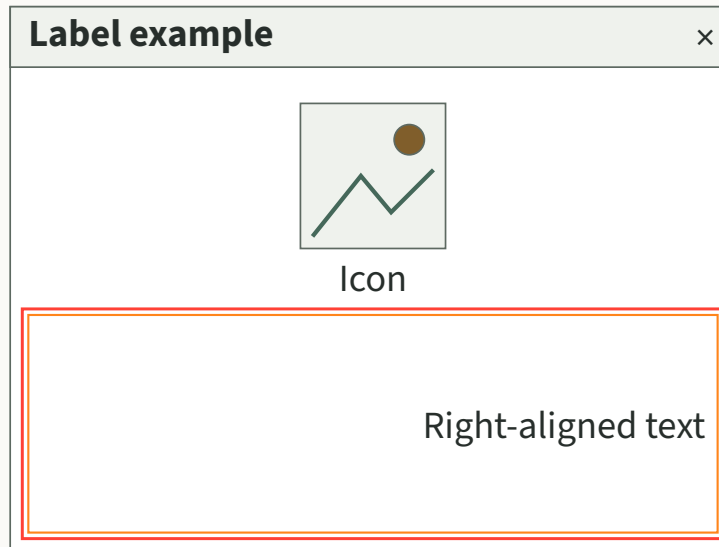


File menu open



Help menu open

JLabel: Text, Images and Borders



← JLabel · image + text

← JLabel · etched border

GridLayout(2, 1) · equal-height rows

Recreate this window with two non-editable labels. Supply your own icon.png.

Configure the Labels

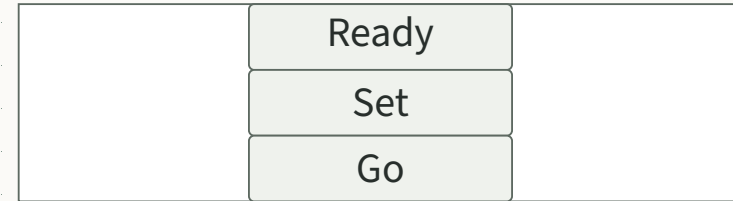
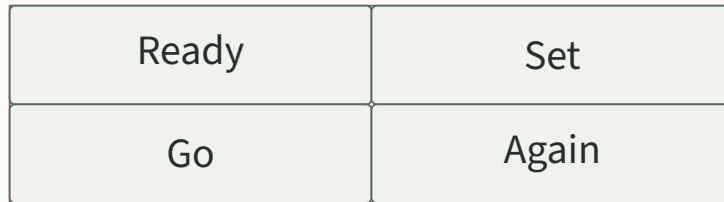
```
1 frame.setLayout(new GridLayout(2, 1));
2 ImageIcon image = new ImageIcon("icon.png");
3 JLabel label1 = new JLabel("Icon", image, JLabel.CENTER);
4 label1.setHorizontalTextPosition(JLabel.CENTER);
5 label1.setVerticalTextPosition(JLabel.BOTTOM);
6 JLabel label2 = new JLabel("Right-aligned text");
7 label2.setHorizontalAlignment(JLabel.RIGHT);
8 label2.setBorder(BorderFactory.createEtchedBorder(
9     Color.RED, Color.ORANGE));
10 frame.add(label1);
11 frame.add(label2);
```

Java

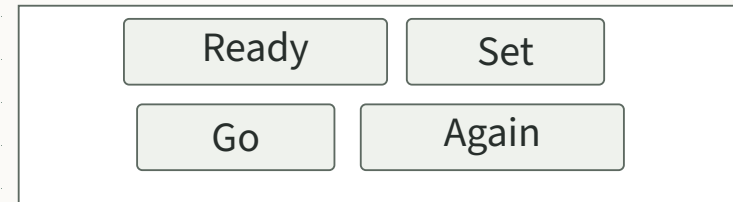
`frame.add(...)` forwards to the content pane. Set the frame size before showing it.

4. Layout

Choose a Layout Manager

BoxLayout.X_AXIS**BoxLayout.Y_AXIS****GridLayout(2, 2)**

Equal-sized cells

FlowLayout

Wrap when the row is full

BoxLayout: Stack Components

```
1 Container contentPane = frame.getContentPane();  
2 contentPane.setLayout(  
3     new BoxLayout(contentPane, BoxLayout.Y_AXIS));  
4 contentPane.add(new JButton("Ready"));  
5 contentPane.add(new JButton("Set"));  
6 contentPane.add(new JButton("Go"));  
7 contentPane.add(new JButton("Again"));  
8 frame.pack();  
9 frame.setVisible(true);
```

Java

`pack()` sizes the window from its components' preferred sizes.

Change the Arrangement

Horizontal row:

```
1 contentPane.setLayout(  
2     new BorderLayout(contentPane, BorderLayout.X_AXIS));
```

Java

Equal-sized cells:

```
1 contentPane.setLayout(new GridLayout(2, 2));
```

Java

Rows that wrap:

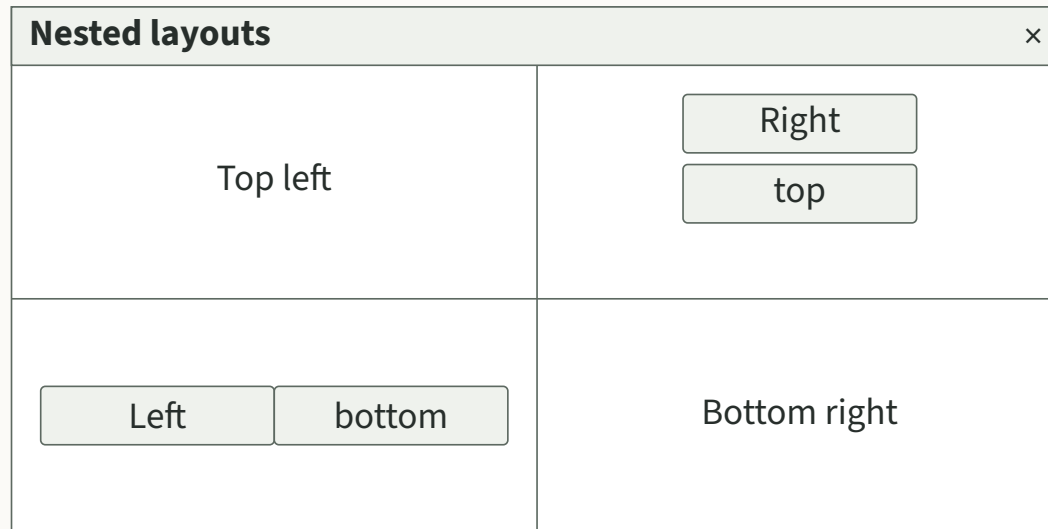
```
1 contentPane.setLayout(new FlowLayout());
```

Java

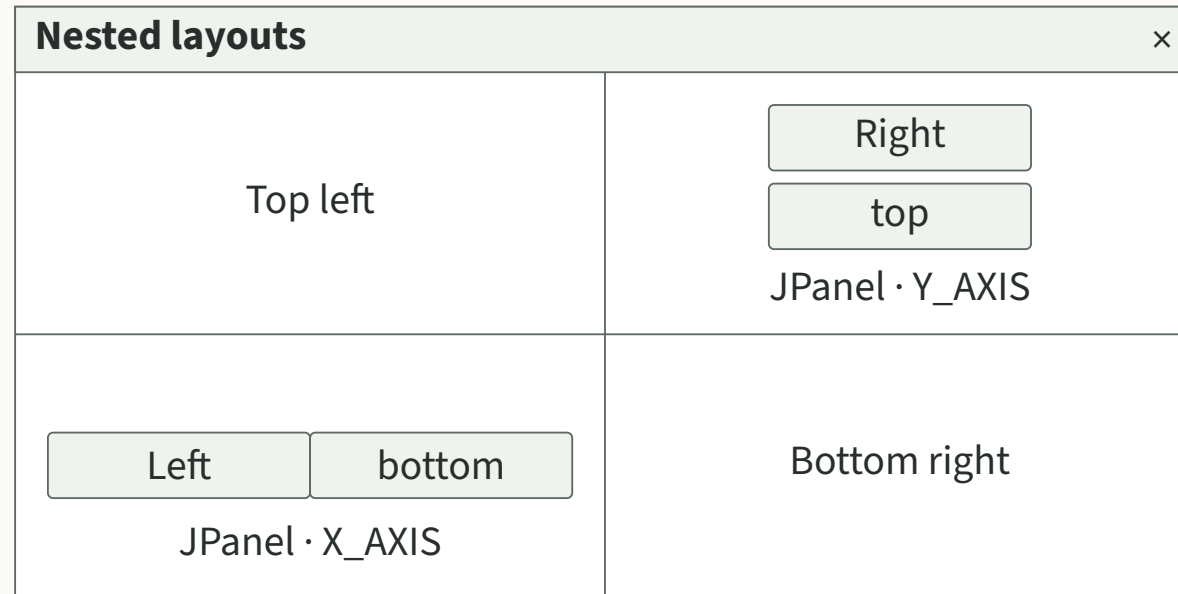
Combine Layouts with JPanel



Which components and layout managers produce this window?



Combine Layouts with JPanel



Content pane · `GridLayout(2, 2)`

Build the Inner Panels

```
1 JPanel panel1 = new JPanel();  
2 panel1.setLayout(new BorderLayout(panel1, BorderLayout.Y_AXIS));  
3 panel1.add(new JButton("Right"));  
4 panel1.add(new JButton("top"));  
5  
6 JPanel panel2 = new JPanel();  
7 panel2.setLayout(new BorderLayout(panel2, BorderLayout.X_AXIS));  
8 panel2.add(new JButton("Left"));  
9 panel2.add(new JButton("bottom"));
```

Java

Place the Panels in the Frame

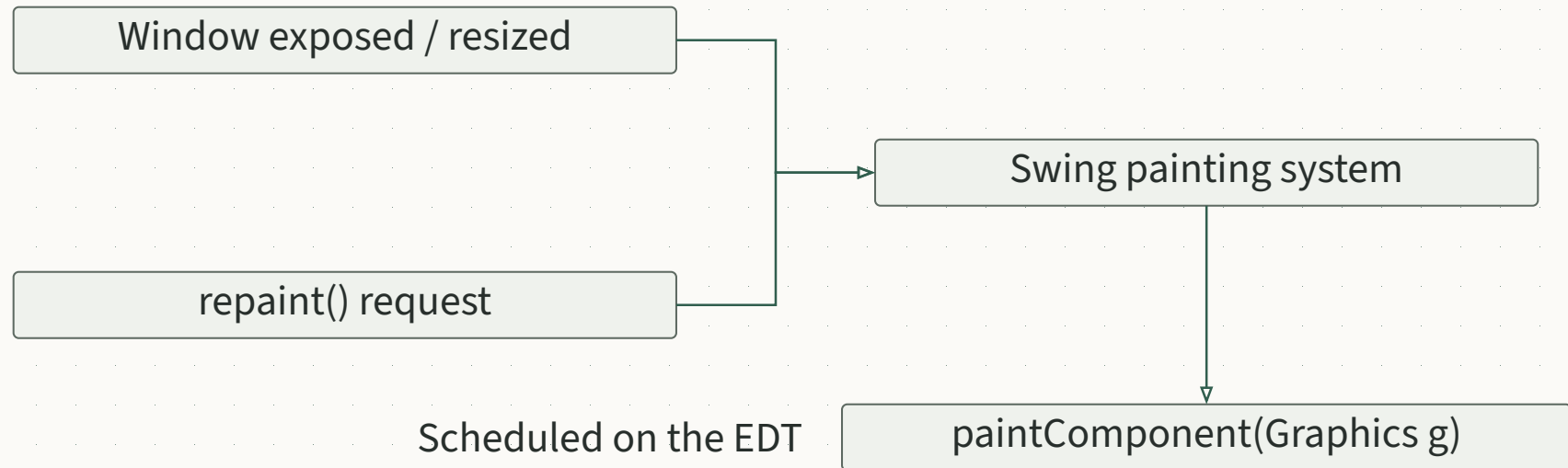
```
1 frame.setLayout(new GridLayout(2, 2));
2 frame.add(new JLabel("Top left", JLabel.CENTER));
3 frame.add(panel1);
4 frame.add(panel2);
5 frame.add(new JLabel("Bottom right", JLabel.CENTER));
6 frame.pack();
7 frame.setVisible(true);
```

Java

Each panel has its own layout manager.

5. Drawing

Swing Controls When Painting Happens



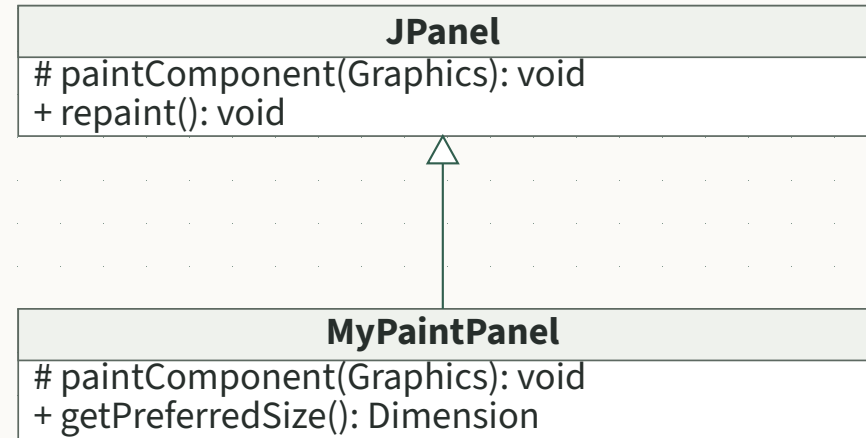
Scheduled on the EDT

Graphics provides text, lines, rectangles, ovals and arcs

`repaint()` schedules a paint; it does not call `paintComponent()` immediately.

Override paintComponent()

```
1 class MyPaintPanel Java
2     extends JPanel {
3     @Override
4     protected void paintComponent(
5         Graphics g) {
6         super.paintComponent(g);
7         // Draw here.
8     }
9 }
```



Override `paintComponent()`; call super first

Drawing Commands Use Pixel Coordinates

Inside `paintComponent()`, after the call to `super`:

```
1  g.setColor(Color.BLACK);
2  g.drawRect(25, 50, 100, 30);
3  g.drawString("JPanel", 55, 70);
4  g.drawRect(25, 120, 100, 30);
5  g.drawString("MyPaintPanel", 40, 140);
6  g.drawLine(75, 80, 75, 120);
7  g.fillPolygon(new int[]{70, 75, 80},
8               new int[]{90, 80, 90}, 3);
9  g.setColor(Color.RED);
10 g.fillRect(202, 70, 12, 42);
11 g.fillOval(200, 120, 16, 16);
```

Java

Give the Drawing a Preferred Size

In MyPaintPanel:

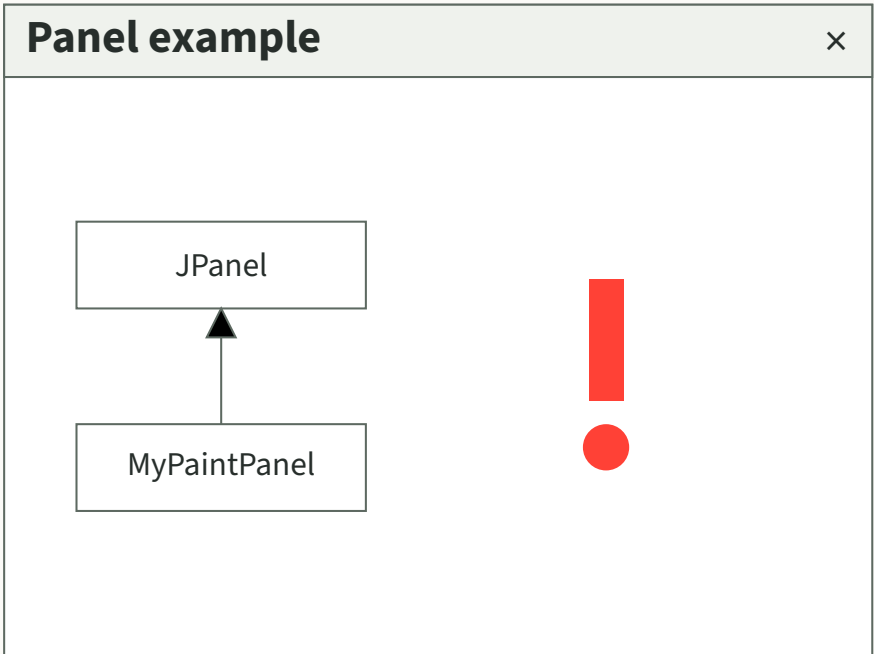
```
1 @Override
2 public Dimension getPreferredSize() {
3     return new Dimension(300, 200);
4 }
```

Java

In the frame's setup:

```
1 frame.add(new MyPaintPanel());
2 frame.pack();
3 frame.setVisible(true);
```

Java

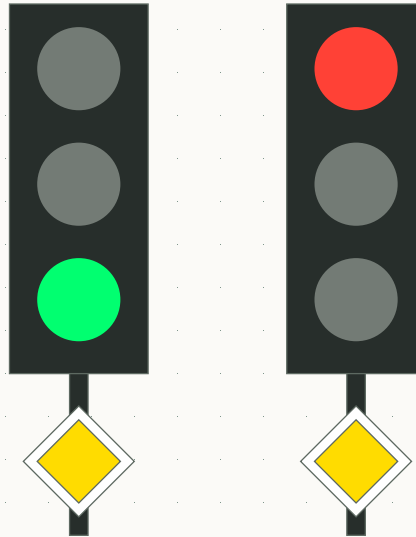


Task: Draw a Traffic Light



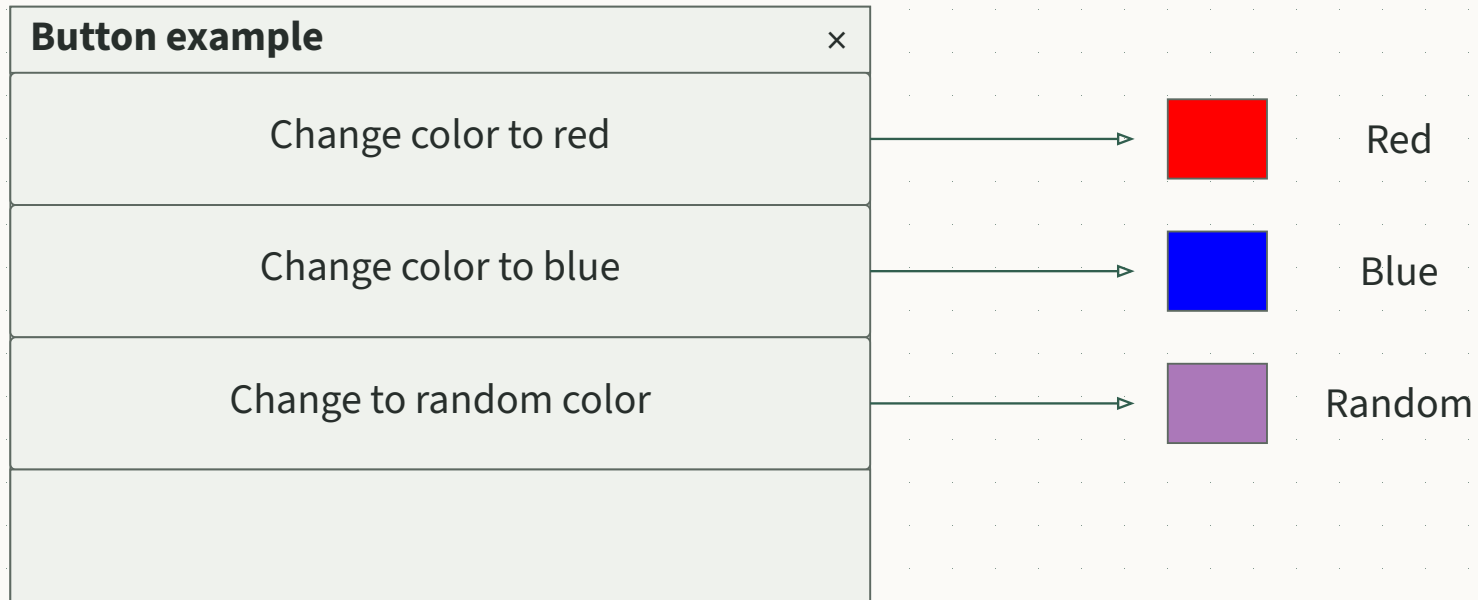
Task 3

Draw a traffic light with rectangles and circles. Choose one of the two states.



6. Buttons & Events

A Click Changes State



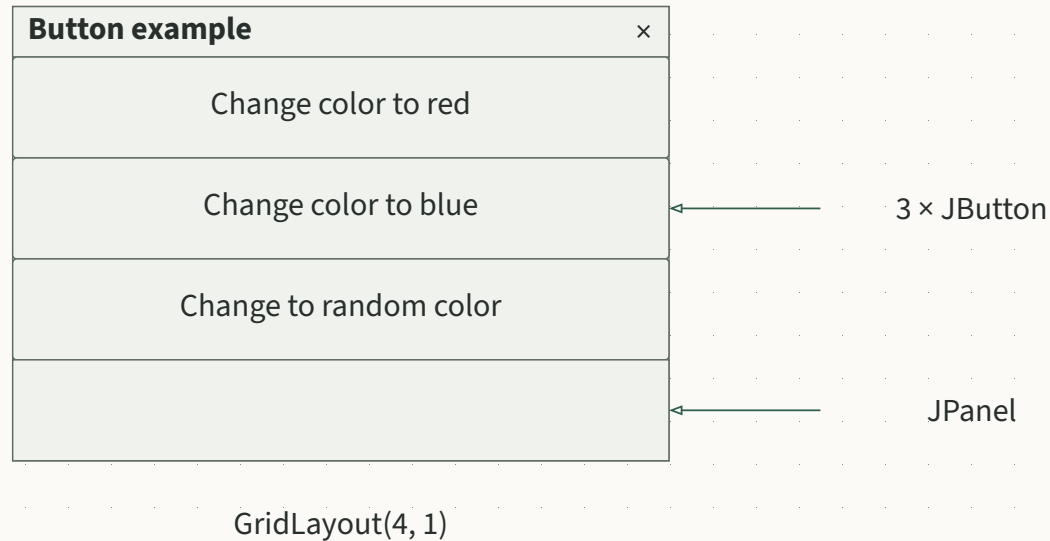
Click → panel background

Build the Interface First



Task 4

Create three buttons and a panel. Arrange them in four equal rows.



Build the Interface First

Fields of ButtonEvent:

```
1 private JPanel panel;  
2 private JButton buttonRed, buttonBlue, buttonRandom;
```

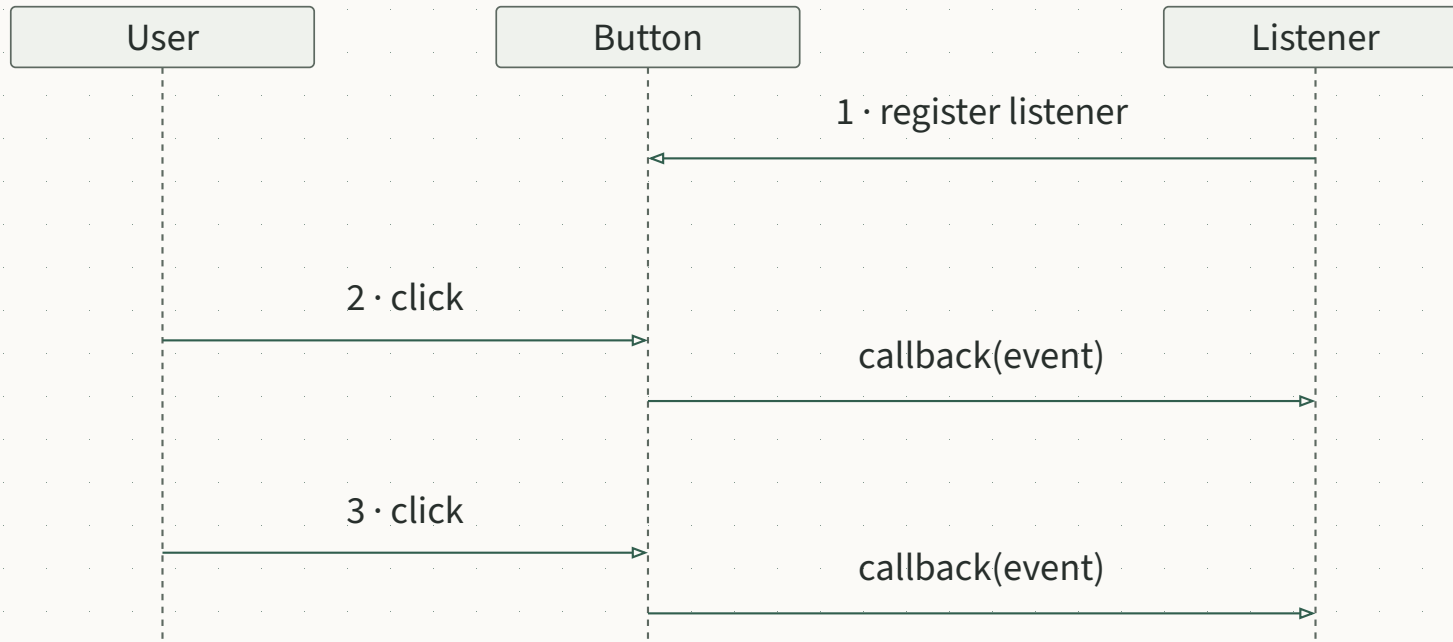
Java

In its constructor, after creating the frame:

```
1 frame.setLayout(new GridLayout(4, 1));  
2 frame.add(buttonRed = new JButton("Change color to red"));  
3 frame.add(buttonBlue = new JButton("Change color to blue"));  
4 frame.add(buttonRandom = new JButton("Change to random color"));  
5 frame.add(panel = new JPanel());  
6 frame.pack();  
7 frame.setVisible(true);
```

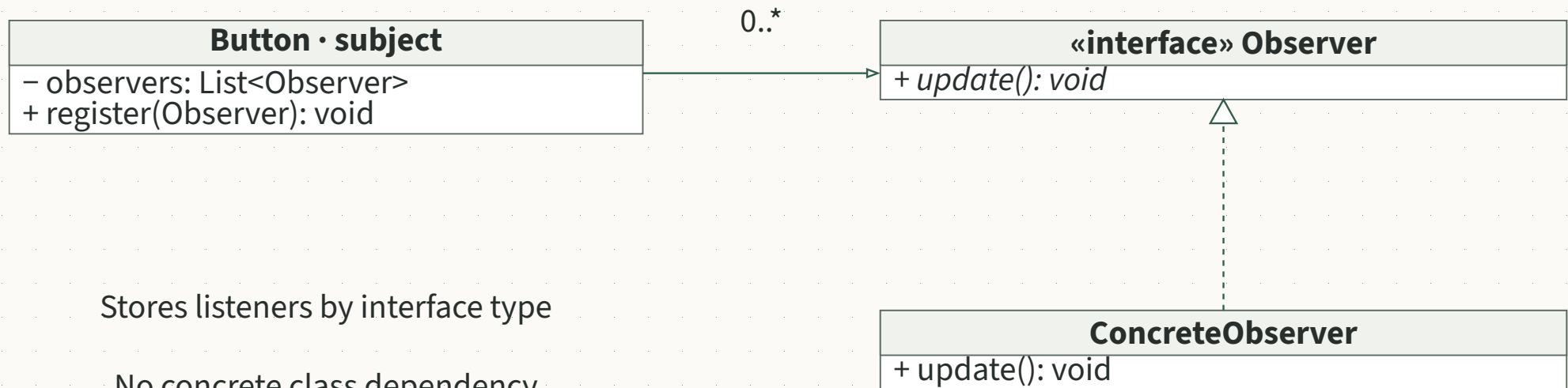
Java

Register Once, React Many Times



Register once · notify on every action · time flows downward

Observer: Depend on a Contract



Stores listeners by interface type

No concrete class dependency

On a change: call `update()` on each registered observer

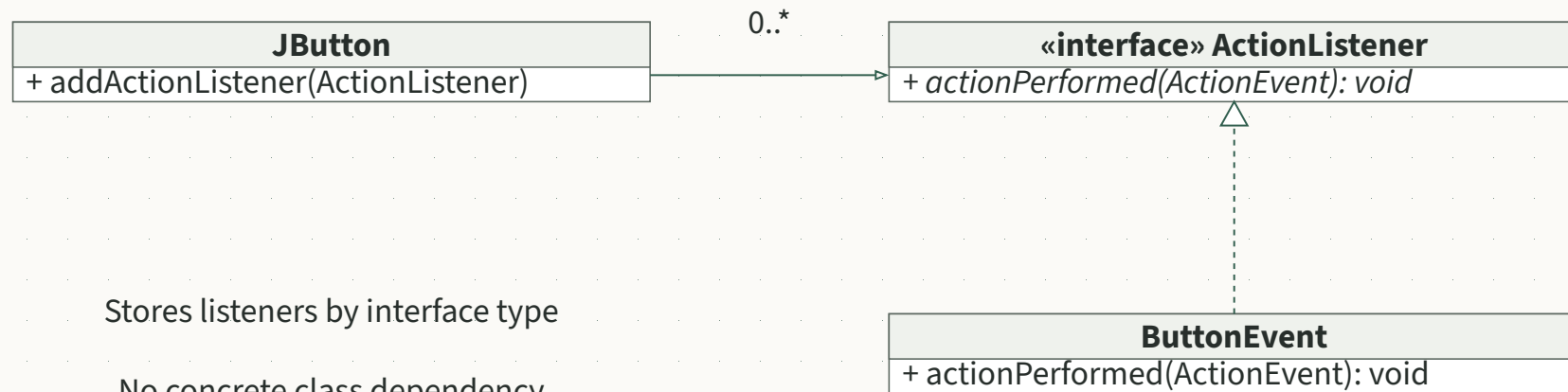
`update()` is a conceptual callback here, not Java's `Object.notify()`.

Swing's Observer Contract



Task 5

Make `ButtonEvent` implement `ActionListener`. Register this object with all three buttons.



On an action: call `actionPerformed(event)` on each registered listener

Register the Listener

```
1 public class ButtonEvent implements ActionListener {  
2     // Fields and constructor as before.  
3     @Override  
4     public void actionPerformed(ActionEvent event) {  
5         // Choose the panel's new color.  
6     }  
7 }
```

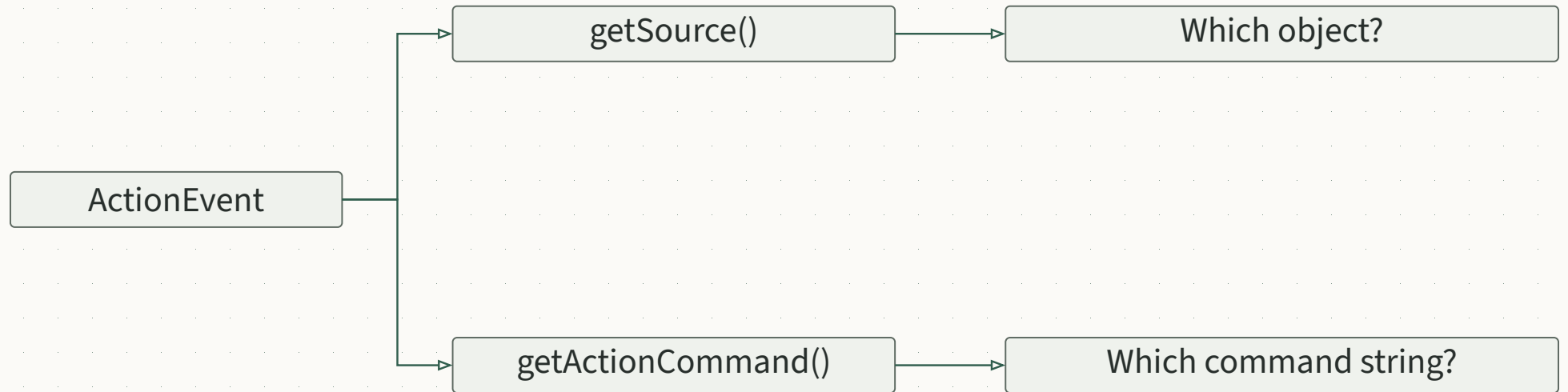
Java

In the constructor, after creating the buttons:

```
1 buttonRed.addActionListener(this);  
2 buttonBlue.addActionListener(this);  
3 buttonRandom.addActionListener(this);
```

Java

Identify the Action



Two ways to choose the response

Respond to the Event Source

```
1  @Override
2  public void actionPerformed(ActionEvent event) {
3      if (event.getSource() == buttonRed) {
4          panel.setBackground(Color.RED);
5      } else if (event.getSource() == buttonBlue) {
6          panel.setBackground(Color.BLUE);
7      } else if (event.getSource() == buttonRandom) {
8          java.util.Random random = new java.util.Random();
9          panel.setBackground(new Color(random.nextFloat(),
10             random.nextFloat(), random.nextFloat()));
11      }
12 }
```

Java

Callbacks run on the EDT. Keep them short so the interface stays responsive.

Alternative: Command Strings

In the constructor:

```
1 buttonRed.setActionCommand("red");  
2 buttonBlue.setActionCommand("blue");  
3 buttonRandom.setActionCommand("random");
```

Java

In actionPerformed():

```
1 String command = event.getActionCommand();  
2 if ("red".equals(command)) {  
3     panel.setBackground(Color.RED);  
4 } else if ("blue".equals(command)) {  
5     panel.setBackground(Color.BLUE);  
6 } else if ("random".equals(command)) {  
7     // Set a random color as before.  
8 }
```

Java

Alternative: One Listener per Button

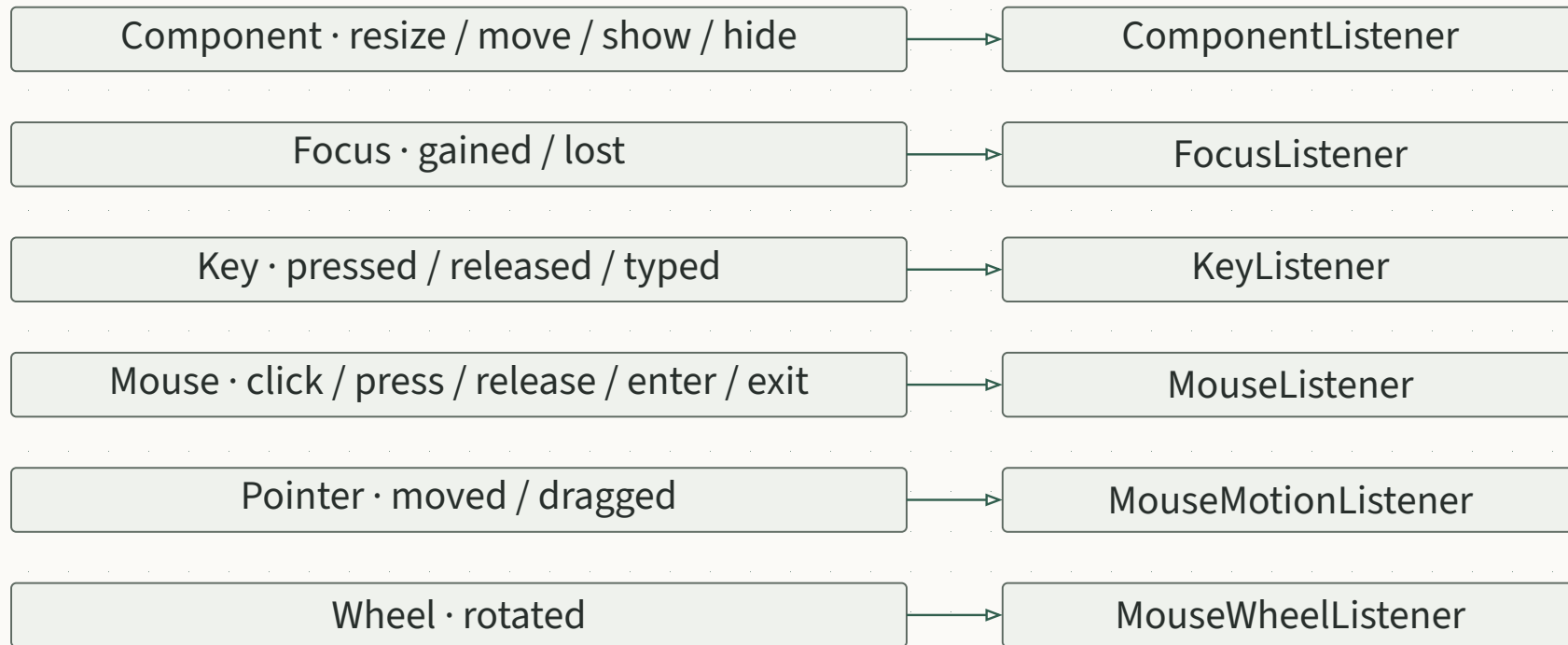
Anonymous class inside the constructor:

```
1 buttonRed.addActionListener(new ActionListener() {  
2     @Override  
3     public void actionPerformed(ActionEvent event) {  
4         panel.setBackground(Color.RED);  
5     }  
6 });
```

Java

The enclosing class no longer needs to implement `ActionListener`. No source check: this listener belongs only to the red button.

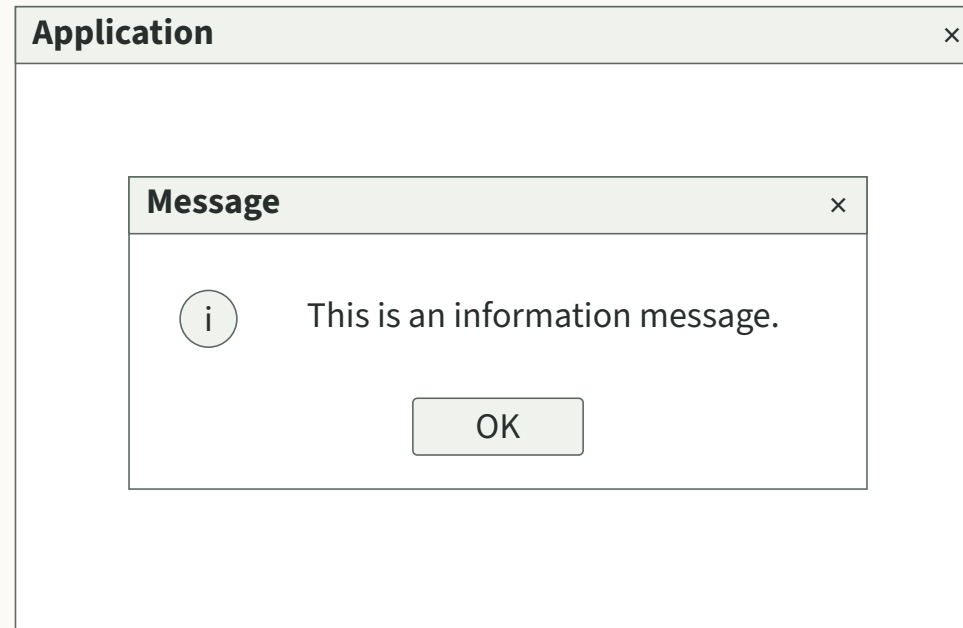
More Events, More Listener Interfaces



KeyListener receives key events only while the component has focus

7. Simple Dialogs

JOptionPane: A Ready-Made Dialog



Modal: dismiss the dialog to return to the parent window

Choose a Message Type

After showing the parent frame, on the EDT:

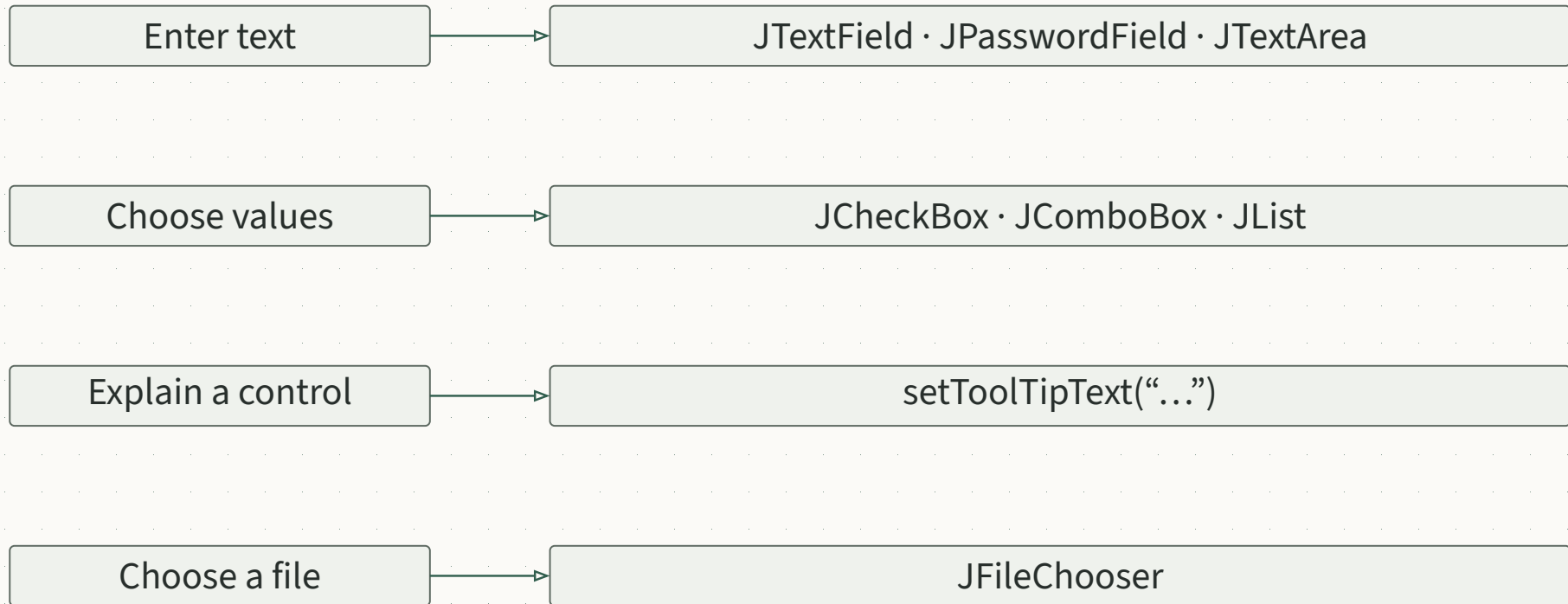
```
1 JOptionPane.showMessageDialog(frame,  
2     "This is a plain message.", "Message",  
3     JOptionPane.PLAIN_MESSAGE);  
4 JOptionPane.showMessageDialog(frame,  
5     "This is an information message.", "Message",  
6     JOptionPane.INFORMATION_MESSAGE);  
7 JOptionPane.showMessageDialog(frame,  
8     "This is a warning.", "Message",  
9     JOptionPane.WARNING_MESSAGE);
```

Java

Each call waits until its dialog is dismissed.

8. Suggestions

Match a Control to the Task



9. License Notice

Attribution

- Shared under CC BY-NC-SA 4.0.
- Based on teaching material by Prof. Dr. Marc Hensel, supplied privately.
- Text, diagrams and layout adapted; the private source cannot be linked.